

Reviewer Pack — Young Tableau Count Inverse Proportional to Monotone Circuit...

Entry 50913375ea21 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 20:27:07 UTC

Young Tableau Count Inverse Proportional to Monotone Circuit Size for 3-CNFs

Entry ID: 50913375ea21 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 20:27:07 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Standard Young Tableaux) **Field B** (complexity object): Monotone Circuit Size for 3-CNFs

Statement:

For any 3-CNF formula with n variables, the number of standard Young tableaux of shape λ (where λ is the partition derived from the formula's clause structure) is $\Theta(1/\text{size}(\text{MC}))$, where $\text{size}(\text{MC})$ is the minimal monotone circuit size required to compute the formula.

Rationale (proposer's reasoning):

The number of standard Young tableaux is connected to the structure of permutations and the combinatorics of the formula. If the formula has a high monotone circuit size, the associated partition λ may have a structure that limits the number of tableaux, suggesting a deeper connection between combinatorial structures and circuit complexity.

Taxonomy category: GCT_DET_PERM (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bf5bd55c545fbbd5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from itertools import combinations

def generate_3cnf(n: int, num_clauses: int) -> list:
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(num_clauses):
        clause = set(random.sample(variables, 3))
        while len(clause) < 3:
            clause.add(random.choice(variables))
        clauses.append(tuple(sorted(clause)))
    return clauses

def rank_matrix(matrix: list[list[int]]) -> int:
    m, n = len(matrix), len(matrix[0])
    for i in range(m):
        max_row = i
        for j in range(i + 1, m):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        if matrix[i][i] == 0:
            return -1
        for j in range(i + 1, m):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    rank = sum(1 for row in matrix if any(row))
    return rank

def hook_length_formula(shape: list[int]) -> int:
    m, n = len(shape), max(shape)
    table = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m):
        for j in range(n):
            if i < shape[j]:
                table[i][j] = (i + 1) * (n - j) // (i + j + 2)
    return int(math.prod(table))
```

```

def generate_partition(clauses: list[tuple[int, ...]]) -> list[int]:
    incidence_matrix = [[0] * len(clauses) for _ in range(2)]
    for i, clause in enumerate(clauses):
        for var in clause:
            if var > 0:
                incidence_matrix[0][i] += 1
            else:
                incidence_matrix[1][i] += 1
    rank = min(rank_matrix(incidence_matrix), len(clauses))
    return [rank]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 20
    num_clauses = 3 * n
    clauses = generate_3cnf(n, num_clauses)
    partition = generate_partition(clauses)
    tableau_count = hook_length_formula(partition)
    circuit_size = len(clauses) # Simplified for testing purposes
    product = tableau_count * circuit_size
    conjecture_holds = product <= 100 # Placeholder value
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "product",
        "metric_value": product,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample,
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = (sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results)) ** 0.5
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_beadacff.py", line 88, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_beadacff.py", line 70, in run_trial
    tableau_count = hook_length_formula(partition)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_beadacff.py", line 51, in hook_length_f
    return int(math.prod(table))
               ^^^^^
```

TypeError: can't multiply sequence by non-int of type 'list'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError, preventing data collection. Support condition cannot be evaluated without successful runs. | next: Fix the hook_length_formula implementation to handle list operations properly and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	41372
2	propose	ollama_remote	qwen3:8b	0	0	93121
3	preregistration	ollama_remote	qwen3:8b	0	0	25106
4	novelty	ollama_remote	qwen3:8b	0	0	20778
5	novelty	ollama_remote	qwen3:8b	0	0	16003
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16338
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10621
8	judge	ollama_remote	qwen3:8b	0	0	16717

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 240056 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in [research/audit/50913375ea21.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/50913375ea21.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/50913375ea21.tar.gz` (if generated)